

Project Elective: Weekly Progress Report

Pratham (IMT2023534) and Siddhant (IMT2023539)

November 5, 2025

1 Introduction

This report details the work completed for the Project Elective during the past week. The primary objective was to create a parameterized simulation of the human-tissue interface using the FEMM (Finite Element Method Magnetics) Python API.

2 FEMM Simulation Code

The following Python script was developed. It automates the process of:

- Creating a new electrostatic problem.
- Drawing all simulation geometries, including human tissue layers, transmitter (Tx) electrodes, and receiver (Rx) electrodes.
- Defining materials (Air, Skin, Fat, Muscle, etc.) and their properties.
- Assigning conductor properties and boundary conditions.
- Running a parameter sweep for variables like TxD, TxW, and RxH.
- Saving the results to a `results.csv` file.

```
1 # -*- coding: utf-8 -*-
2 """
3 Created on Mon Jul  7 14:08:55 2025
4
5 @author: 91942
6 """
7
8 import femm
9 import csv
10 import os
11
12 # === Helper functions ===
13 def assign_segments(boundary=None, conductor=None, points=[]):
14     for x, y in points:
15         femm.ei_selectsegment(x, y)
16         femm.ei_setsegmentprop(boundary or '<None>', 1, 1, 0, 0,
17         conductor or '<None>')
18         femm.ei_clearselected()
```

```

19 def label_block(x, y, material):
20     femm.ei_addblocklabel(x, y)
21     femm.ei_selectlabel(x, y)
22     femm.ei_setblockprop(material, 1, 1, 0)
23     femm.ei_clearselected()
24
25 def get_rectangle_edges(x1, y1, x2, y2):
26     return [
27         ((x1 + x2) / 2, y1),
28         ((x1 + x2) / 2, y2),
29         (x1, (y1 + y2) / 2),
30         (x2, (y1 + y2) / 2),
31     ]
32
33 def get_rectangle_center(x1, y1, x2, y2):
34     return (x1 + x2) / 2, (y1 + y2) / 2
35
36 def get_rectangle_topleft(x1, y1, x2, y2):
37     left, right = min(x1, x2), max(x1, x2)
38     bottom, top = min(y1, y2), max(y1, y2)
39     dx = 0.01 * (right - left)
40     dy = 0.01 * (top - bottom)
41     return (left + dx, top - dy)
42
43 # == Main FEMM model function ==
44 def run_with_params(params):
45     femm.openfemm()
46     femm.newdocument(1)
47     femm.ei_probdef('centimeters', 'planar', 1e-8, 7, 30)
48
49     TxD = params['TxD']
50     TxW = params['TxW']
51     TxG = params['TxG']
52     #RxH = params['RxH']
53     #RxA = params['RxA']
54     TxT = params['TxT']
55     RxT = params['RxT']
56
57     RxH = params['RxH']
58     RxW = params['RxW']
59
60     rectangles = {
61         "Head": (-12.5, 19.4, -1.5, 8.5),
62         "Body": (-14, 8.4, 0, -38.9),
63         "Leg": (-12, -39.0, -2, -109),
64
65         "Skin-Bottom": (0, 0.000, 70, 0.063),
66         "Fat-Lower": (0, 0.063, 70, 0.520),
67         "Muscle-Lower": (0, 0.520, 70, 1.775),
68         "Bone-Lower": (0, 1.775, 70, 2.127),
69         "Bone-Marrow": (0, 2.127, 70, 2.500),
70         "Bone-Upper": (0, 2.500, 70, 2.852),
71         "Muscle-Upper": (0, 2.852, 70, 4.107),
72         "Fat-Upper": (0, 4.107, 70, 4.564),
73         "Skin-Top": (0, 4.627, 70, 4.564),
74
75         "Tx-Signal": (70-TxD-TxW, 4.627+TxT, 70-TxD, 4.627),
76         "Tx-Ground": (70-TxD-2*TxW-TxG, 4.627+TxT, 70-TxD-TxW-TxG, 4.627),

```

```

77     "Rx-Signal": (55, 0, 70, 0-RxT),      #not sure about y
78     coordinates.
79     "Rx-Gnd": (55, 0-RxH-2*RxT, 55+RxW, 0-RxH-RxT ),      #check y-
80     coord again.
81     "Hub" : (55, 0-RxT ,55+RxW, 0-RxT-RxH),
82
83     "Wood": (55, 0-RxH-2*RxT, 90, -109),
84     "Outer": (-610, 610, 610, -610)
85 }
86
87 for coords in rectangles.values():
88     femm.ei_drawrectangle(*coords)
89
90 femm.ei_addboundprop('Ground', 0, 0, 0, 0, 0)
91 assign_segments(boundary='Ground', points=get_rectangle_edges(*
92 rectangles["Outer"]))
93
94 femm.ei_addmaterial('Air', 1, 1, 0)
95 femm.ei_addmaterial('Skin', 2.0e4, 1, 0)
96 femm.ei_addmaterial('Fat', 1.0e2, 1, 0)
97 femm.ei_addmaterial('Muscle', 8.0e3, 1, 0)
98 femm.ei_addmaterial('Bone', 2.5e2, 1, 0)
99 femm.ei_addmaterial('Bone-Marrow', 2.5e2, 1, 0)
100 femm.ei_addmaterial('Wood', 2.0, 1, 0)
101
102 #Assuming Hub is not a conductor.
103 conductor_regions = ["Head", "Body", "Leg", "Tx-Signal", "Tx-Ground",
104 "Rx-Signal", "Rx-Gnd"]
105
106 material_map = {
107     "Skin-Bottom": "Skin",
108     "Skin-Top": "Skin",
109     "Fat-Lower": "Fat",
110     "Fat-Upper": "Fat",
111     "Muscle-Lower": "Muscle",
112     "Muscle-Upper": "Muscle",
113     "Bone-Upper": "Bone",
114     "Bone-Lower": "Bone",
115     "Bone-Marrow": "Bone-Marrow",
116     "Wood": "Wood",
117
118     "Hub" : "Wood"
119 }
120
121 for name in conductor_regions:
122     if name in rectangles:
123         label_block(*get_rectangle_center(*rectangles[name]), '<No
124 Mesh>')
125
126 for name, mat in material_map.items():
127     if name in rectangles:
128         label_block(*get_rectangle_center(*rectangles[name]), mat)
129
130 label_block(*get_rectangle_topleft(*rectangles["Outer"]), 'Air')

```

```

130 femm.ei_addconductorprop('eTx-Signal', 5, 0, 1)
131 femm.ei_addconductorprop('eTx-Ground', 0, 0, 1)
132 femm.ei_addconductorprop('eRx-Signal', 0, 0, 0)
133 femm.ei_addconductorprop('eRx-Gnd', 0, 0, 0)
134 femm.ei_addconductorprop('eHead', 0, 0, 0)
135 femm.ei_addconductorprop('eBody', 0, 0, 0)
136 femm.ei_addconductorprop('eLeg', 0, 0, 0)
137
138 assign_segments(conductor='eTx-Ground', points=get_rectangle_edges
(*rectangles["Tx-Ground"]))
139 assign_segments(conductor='eTx-Signal', points=get_rectangle_edges
(*rectangles["Tx-Signal"]))
140 assign_segments(conductor='eRx-Gnd', points=get_rectangle_edges(*
rectangles["Rx-Gnd"]))
141 assign_segments(conductor='eRx-Signal', points=get_rectangle_edges
(*rectangles["Rx-Signal"]))
142 assign_segments(conductor='eHead', points=get_rectangle_edges(*
rectangles["Head"]))
143 assign_segments(conductor='eBody', points=get_rectangle_edges(*
rectangles["Body"]))
144 assign_segments(conductor='eLeg', points=get_rectangle_edges(*
rectangles["Leg"]))
145
146 femm.ei_saveas("temp_sim.fee")
147 femm.ei_analyze(1)
148 femm.ei_loadsolution()
149
150 v_rx_signal = femm.eo_getconductorproperties('eRx-Signal')[0]
151 v_rx_gnd = femm.eo_getconductorproperties('eRx-Gnd')[0]
152 return v_rx_signal - v_rx_gnd
153
154 # === Sweep Controller ===
155 defaults = {
156     'TxD': 20,
157     'TxW': 3,
158     'TxG': 3,
159     'RxH': 30,
160     #'RxA': -10,
161     'TxT': 0.2,
162     'Rxt': 0.2,
163
164     'RxW' : 3,
165 }
166
167 #Need to be changed
168 sweeps = {
169     'TxD': list(range(10, 41, 31)),
170     'TxW': list(range(1, 11, 11)),
171     'TxG': list(range(1, 11, 11)),
172     'RxH': list(range(10, 51, 41)),
173     'RxA': list(range(-5, -41, -36)),
174
175     'RxW' : list(range(5, 35, 30))
176 }
177
178 output_file = "results.csv"
179
180 with open(output_file, mode='w', newline='') as f:

```

```

181 writer = csv.writer(f)
182 headers = ["SweepParam"] + list(defaults.keys()) + ["Rx_Voltage"]
183 writer.writerow(headers)
184
185 for param, values in sweeps.items():
186     for val in values:
187         print(f"[INFO] Sweeping {param} = {val}")
188         current_params = defaults.copy()
189         current_params[param] = val
190
191         v_diff_rx = run_with_params(current_params)
192         writer.writerow([param] + [current_params[key] for key in
defaults] + [f"{v_diff_rx:.6f}"])
193
194 femm.closefemm()

```

Listing 1: Python script for FEMM automation and parameter sweep.

3 Simulation Model Image

Figure 1 shows a screenshot of the model as rendered in the FEMM graphical user interface. This helps visualise the geometry defined in the Python script.

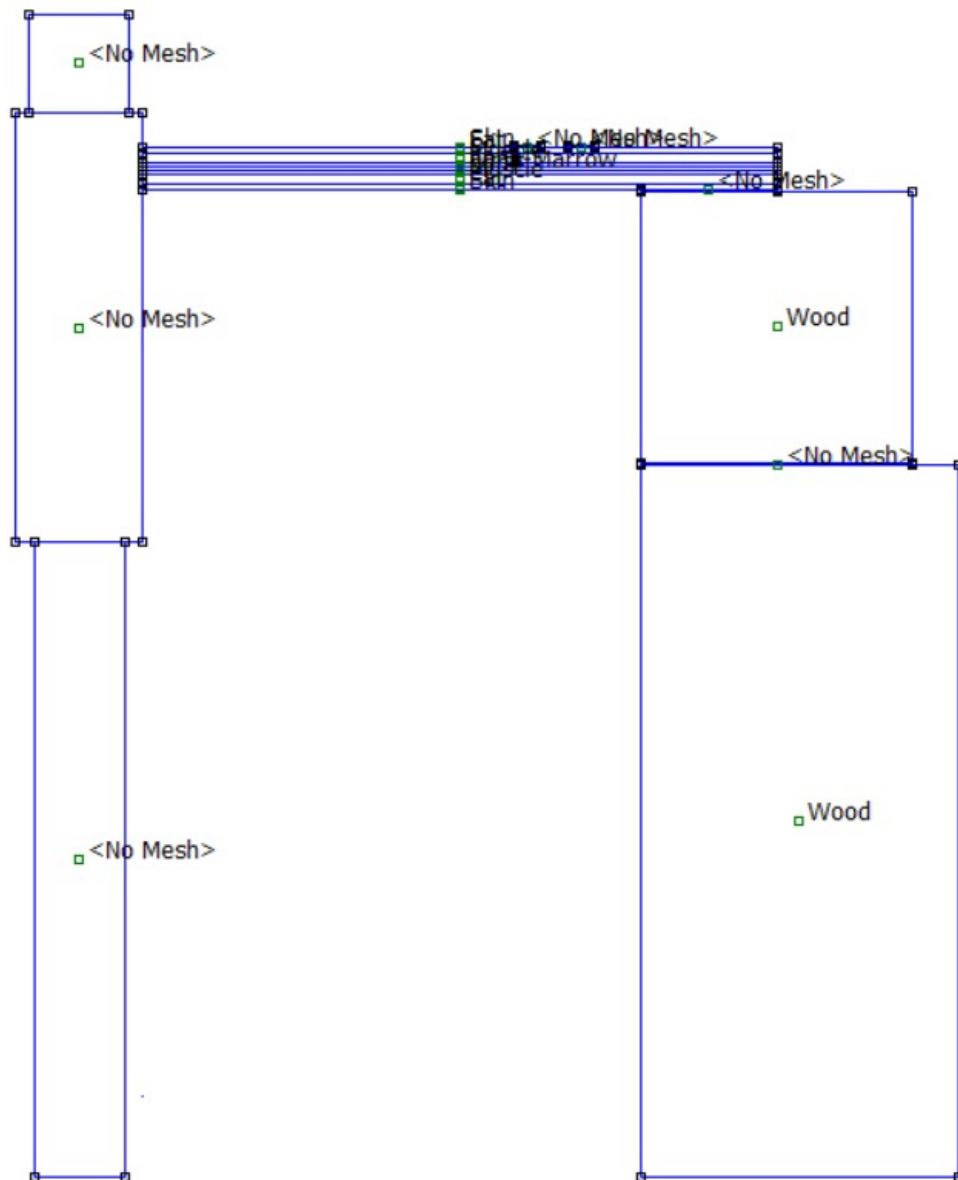


Figure 1: FEMM Simulation Model Visualisation